Metamaterials: Application

Lecture 4 – Integrating deep learning into lattice design

Dr. L. Lomazzi

April 21, 2026

Roadmap

1. **Part A** – The lattice design problem, from a DL perspective
2. **Part B** – Forward surrogates for lattices
3. **Part C** – Generative models for lattice design
4. **Part D** – Inverse design
5. **Part E** – End-to-end workflow
6. **Part F** – Wrap-up

Bridge from Lecture 3

Design space (trusses / TPMS / Voronoi) + target $\sigma(\varepsilon)$ – now use the morning's DL toolbox to navigate it.

L4 – What we want DL to do for us

Classical workflow. Pick a topology, pick $\bar{\rho}$, simulate, iterate, hope.

Three limits.

- **Slow** – each candidate = one explicit-dynamics FE run.
- **Narrow** – topology from a small catalogue.
- **Hard to invert** – "target $\sigma^{\star}_{\mathrm{pl}}, \varepsilon_d$" has no closed form.

Three DL roles

Surrogate $\rightarrow$ *speed*. Generator $\rightarrow$ *exploration*. Inverse design $\rightarrow$ *invertibility*.

L4 – Representations of a lattice

How a lattice looks to a NN depends on encoding:

- **(a) Parametric.** $\boldsymbol{\alpha} \in \mathbb{R}^k$, $k \sim 5$–50. Simplest; fixed family.
- **(b) Voxel.** $\boldsymbol{\rho} \in [0,1]^{N^3}$. Topology-agnostic; high-dim.
- **(c) Graph.** Nodes = strut junctions, edges = struts. Natural for trusses.

Choice drives architecture

Parametric → MLP | Voxel → 3D CNN / U-Net | Graph → GNN

(a) parametric

$$[\alpha_1,\ \alpha_2,\ \ldots,\ \alpha_k]$$

(b) voxel

(c) graph

L4 – Three DL blocks, mapped onto lattices

Block	**What it predicts**	**Lattice use**
Surrogate (L1)	$\sigma^{\star}_{\mathrm{pl}}, \varepsilon_d, \mathrm{SEA}, \sigma(\varepsilon)$	fast screening in loops
Generator (L2)	distribution of valid lattices	propose matching candidates
Inverse (L2)	design from target	direct target→design

They compose

Generator proposes K candidates $\rightarrow$ surrogate ranks $\rightarrow$ best FE-validated $\rightarrow$ feedback.

L4 – The forward map

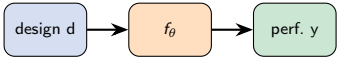
Goal. A fast function

$$f_\theta : \mathrm{d} \longmapsto \mathrm{y}, \quad \text{replaces one FE run.}$$

Input d: parametric, voxel, or graph.

Output y:

- *Scalars*: $(\sigma^\star_{\mathrm{pl}}, \varepsilon_d, \mathrm{SEA}, E^\star, \dots)$.
- *Full curve*: σ at N_ε strain points.



"one FE run" in milliseconds

Practical tip

Start with scalars. Go to full curves only if you need the curve *shape*.

L4 – Architecture for each representation

	Input	**Architecture**	**Why**
Parametric	$\boldsymbol{\alpha} \in \mathbb{R}^k$	MLP	low-dim, tabular
Voxel	$\boldsymbol{\rho} \in [0,1]^{N^3}$	3D CNN / U-Net	spatial locality
Graph	nodes + edges	GNN	mesh-agnostic

Rule of thumb

Match architecture to representation. MLP on voxels needs millions of examples.

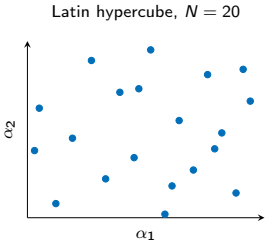
L4 – Building the dataset (the real bottleneck)

Size target. $N \sim 10^3$–10^5 labeled designs.

Each label = one explicit-dynamics FE run ($\sim$ 10–60 min on a workstation).

Sampling strategies.

- Latin-hypercube (baseline).
- Halton / Sobol (better for $k \geq 5$).
- Active learning (next).

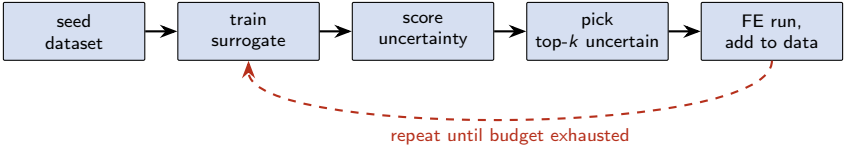


Notes

- k = dim. of design vector $\boldsymbol{\alpha}$.
- Halton / Sobol are *quasi-random, low-discrepancy* sequences – fill $[0,1]^k$ more uniformly than random sampling, and (unlike LHS) can be extended with more points later.

L4 – Active learning

Idea. Sample where the surrogate is *most uncertain*, not uniformly.



Uncertainty estimates: ensemble variance | MC dropout.

MC Dropout: leave dropout on at inference; run the net T times; use the variance across runs as uncertainty

Payoff

Typically 3–10$\times$ fewer FE runs for the same accuracy.

L4 – Physics-aware surrogate tricks

Physics can be leveraged to make training easier. Raw MSE already works, but physics gives free leverage.

1. **Gibson–Ashby prior.** Parameterize $\sigma^\star_{\mathrm{pl}} = C(\boldsymbol{\alpha})\,\bar{\rho}^{n(\boldsymbol{\alpha})}$; learn C, n. Correct asymptotics, even on unseen $\bar{\rho}$.
2. **Output constraints.** $\sigma^\star_{\mathrm{pl}} > 0$, $0 < \varepsilon_d < 1$ via softplus / sigmoid head.
3. **Consistency losses.** Penalize $|\mathrm{SEA}_{\mathrm{pred}} - \sigma^\star_{\mathrm{pl,pred}}\,\varepsilon_{d,\mathrm{pred}}/\rho^\star|$.

Why it matters

These often turn a 1k-design dataset into the equivalent of a 10k-design one.

L4 – Validating a lattice surrogate

ML models need to be validated before they can be used. On a random hold-out:

- **Extrapolation slice.** Hold out $\bar{\rho} > 0.35$; check scaling still holds. Extrapolation allowed as we included physics.
- **Topology transfer.** Train on BCC + octet; test on Kelvin.
- **Scaling-law fit.** Fit Gibson–Ashby n to predictions; compare to theory.
- **Failure-mode check.** Does it flag fracturing lattices, not just predict numbers?

Trust gating

The surrogate's role (screening vs. design decisions) should scale with how well it passes these checks, not with its average error.

L4 – Why generative?

Surrogate answers "is this good?" — Generator answers "give me a good one."

- **Feasibility prior.** Train on valid lattices $\Rightarrow$ samples are plausible.
- **Continuous morphing.** Interpolate between BCC and octet; see intermediates.
- **Conditional proposals.** Sample directly from a performance target.
- **Compression.** 10^5–10^8 voxels $\rightarrow \sim$ 10–100 latent coordinates.

The compression point

Voxel-space inverse design is hopeless. 32-dim latent space is tractable.

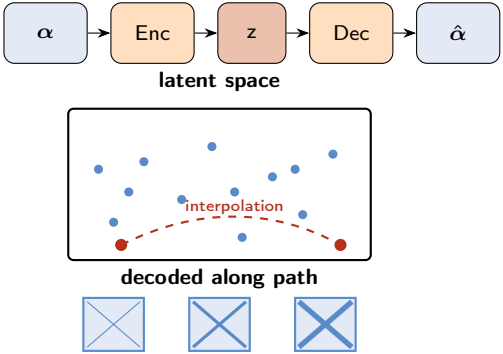
L4 – VAE for parametric lattices

Setup.

- Unit cell described by $\boldsymbol{\alpha} \in \mathbb{R}^k$, $k \sim 10$–50 (strut thicknesses, node offsets, $\bar{\rho}$, ...).
- VAE is MLP-based (no spatial structure).
- Trained on a library of valid $\boldsymbol{\alpha}$'s $\Rightarrow$ decoder learns the feasibility manifold.

What the latent z buys us.

- Dimension $k' \leq k$ – modest compression but more structured.
- Sampling $\mathrm{z} \sim \mathcal{N}(0, \mathrm{I}) \rightarrow$ valid new cells respecting training correlations.
- Smooth interpolation between two known cells.



Limits. Stuck inside the family $\boldsymbol{\alpha}$ describes; inherits training-set biases

L4 – 3D voxel VAE

More ambitious: let the network *see topology as geometry*.

- Each sample: 64^3 binary voxel cell.
- Encoder: 3D CNN
- Decoder: 3D transposed-CNN.
- Latent: $z \in \mathbb{R}^{32}$.

What emerges. Latent dims *often* organize around interpretable features (density, beam angle, connectivity) – with no supervision. BCC $\leftrightarrow$ gyroid interpolations are smooth and manufacturable.

Why powerful

Not tied to a topology family. Explores the full shape space that "looks like" training data.

L4 – GANs and diffusion

GANs.

- Sharper voxels than VAEs.
- **Mode collapse** is real.
- Use WGAN / StyleGAN variants.

Mode collapse: G produces only a few topology types, ignoring the rest of the training distribution.

Diffusion.

- SOTA for 3D shapes (2026).
- Stable, diverse, strong conditioning.
- 50–1000 denoising steps.
- **Latent diffusion** cuts to a handful.

Latent diffusion: run the denoising in a low-dim latent space, then decode to voxels $\Rightarrow \sim$ 10–50 steps instead of 1000.

Practical recommendation

Start with a voxel VAE. Move to diffusion when sharpness or conditioning strength is critical.

L4 – Conditioning on performance

Adding target performance to a generator turns it into a design tool.

cVAE with y entering both encoder and decoder:

$$q_\phi(\mathrm{z} \mid \mathrm{d}, \mathrm{y}), \qquad p_\psi(\mathrm{d} \mid \mathrm{z}, \mathrm{y}).$$

At inference. Fix $\mathrm{y} = \mathrm{y}^\star$, sample z_k, decode:

$$\mathrm{d}_k = D(\mathrm{z}_k, \mathrm{y}^\star), \quad k = 1, \ldots, K.$$

K diverse candidates, all predicted to hit $\mathrm{y}^\star$.

Payoff

User specifies target → gets a family of lattices → picks by secondary criteria.
Milliseconds per candidate.

L4 – Evaluating generated lattices

Three axes, in order:

- **Validity.** Connected? Manufacturable?
- **Performance (predicted).** Run the surrogate on every generated sample.
- **Diversity.**
  - *Pairwise distances* in a reference latent space – checks whether samples are spread or clustered.
  - *Topology-classifier accuracy* – does a CNN classifier see all families, in the right proportions?

The validation ladder

Generate $\rightarrow$ filter invalids $\rightarrow$ surrogate score $\rightarrow$ top-k FE. Each step filters $\sim 10\times$.

L4 – The lattice inverse problem

Given. Target $\mathrm{y}^\star$ (or $\sigma^\star(\varepsilon)$).
Find. Manufacturable d with $F(\mathrm{d}) \approx \mathrm{y}^\star$.

Difficulties.

- High dimension (voxels: 10^5–10^8).
- Non-unique (many d $\to$ same $\mathrm{y}^\star$).
- Non-differentiable physics (contact, fracture, rate effects).
- Manufacturing constraints.

Three strategies

A. Gradient through surrogate. B. Latent-space optimization. C. Conditional generation.

L4 – Strategy A: gradient through the surrogate

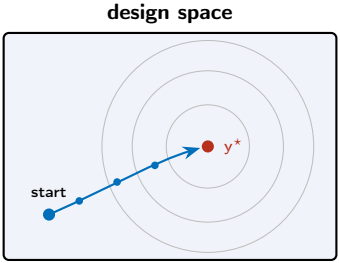
Idea. Treat the design d itself as the optimization variable. Use the surrogate f_θ as a *differentiable* replacement for the FE solver, and do gradient descent on d directly.

The loss.

$$\min_{\mathrm{d}} \underbrace{\|f_\theta(\mathrm{d}) - \mathrm{y}^\star\|^2}_{\text{hit the target}} + \lambda \underbrace{\mathcal{R}(\mathrm{d})}_{\text{stay feasible}}$$

How it works.

- Start from a random / nominal d.
- Compute gradient $\nabla_{\mathrm{d}}\mathcal{L}$ via autograd through f_θ.
- Step: $\mathrm{d} \leftarrow \mathrm{d} - \eta\, \nabla_{\mathrm{d}}\mathcal{L}$.
- Repeat until convergence.



Each step: take a gradient of the surrogate. No FE in the loop.

L4 – Strategy B: latent-space optimization

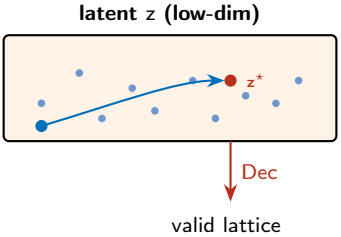
Idea. Don't optimize the design directly. Optimize a *short latent code* z inside a pre-trained VAE, then decode it. The decoder only produces valid-looking lattices ⇒ the optimizer can't wander into nonsense regions.

The loss.

$$\min_{z} \; \|f_\theta(\mathrm{Dec}(z)) - y^\star\|^2 + \lambda \|z\|^2$$

Why it beats Strategy A.

- **Low dim**: $z \in \mathbb{R}^{32}$, not $\mathbb{R}^{10^5}$.
- **Built-in feasibility**: the decoder was trained only on valid lattices.
- **Smoother landscape**: local minima tend to be physical.



Optimize a short code; decoder reconstructs a valid design every time.

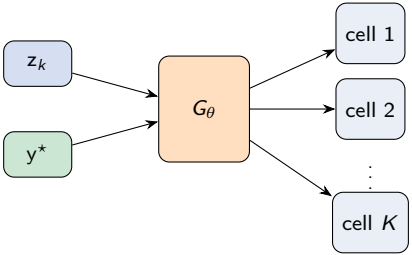
L4 – Strategy C: direct conditional generation

Idea. No optimization at all at inference. A trained *conditional* generator takes the target $\mathrm{y}^\star$ as input and spits out lattices directly.

The formula.

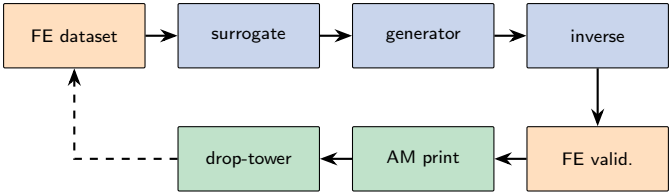
$$\mathrm{d}_k \;=\; G_\theta(\mathrm{z}_k,\ \mathrm{y}^\star), \qquad \mathrm{z}_k \sim \mathcal{N}(0, \mathrm{I}).$$

- Sample K random noise vectors $\mathrm{z}_1, \ldots, \mathrm{z}_K$.
- Pass each through G_θ together with the target.
- K diverse candidates, all predicted to hit $\mathrm{y}^\star$.
- Cost: milliseconds per candidate.



One forward pass per candidate. No iterative optimization.

L4 – The full workflow



Orange: physics. **Blue**: DL. **Green**: hardware. **Dashed**: experimental feedback.

The point

Every stage is **amortized**. Trained DL answers in seconds; FE and experiments only validate.

L4 – Manufacturing in the loop

1. **Post-hoc filter.** Generate freely, reject unbuildable. Simple, wasteful.
2. **Printability loss.** Penalize overhangs, thin features, enclosed voids during training.
3. **Constrained representation.** TPMS level-sets (self-supporting), or parameters that can't produce infeasible geometry. Strongest guarantee.

Recommendation

Combine (2) + (3): constrain the representation, soft-penalize edge cases.

L4 – Validation: never skip FE

Every DL-proposed candidate has non-trivial probability of being wrong.

What fails. Surrogate extrapolation | decoder artifacts | as-printed $\neq$ as-designed.

Two-stage validation.

1. High-fidelity FE on top-k – rates, contact, failure. Hours each.
2. Experiment on top 1–3.

Calibration

Track discrepancy *surrogate* $\rightarrow$ *HF FE* $\rightarrow$ *experiment*. Each layer's error re-trains the previous.

L4 – Multiple objectives

Real lattices are never one-objective: high SEA *and* high $E^\star$ *and* low mass *and* printability.

1. **Weighted sum.** Simple; weight tuning painful.
2. **Pareto frontier.** Sweep targets, map achievable envelope, let engineer pick the knee.
3. **Constraint satisfaction.** Minimize mass s.t. $\mathrm{SEA} \geq \mathrm{SEA}_{\mathrm{min}}$, $E^\star \geq E^\star_{\mathrm{min}}$.

Practical

(2) is most informative; (3) cleanest when requirements are non-negotiable. DL makes both tractable – candidates are cheap.

L4 – Open challenges (2026)

- **Data efficiency.** Dynamic FE is expensive $\rightarrow$ transfer learning, multi-fidelity.
- **Full-curve matching.** Match shape of $\sigma(\varepsilon)$, not just scalars $\rightarrow$ neural operators.
- **Manufacturing noise.** As-printed $\neq$ as-designed $\rightarrow$ robust optimization.
- **Multi-scale design.** Cell $\times$ grading $\times$ shape $\rightarrow$ hierarchical generators.
- **Certification.** Who signs off on a DL-designed aerospace part?

L4 – Summary

- **Representation first**, architecture follows.
- **Surrogate** + active learning + physics priors $\Rightarrow$ fast.
- **Generator** $\Rightarrow$ feasibility prior, compressed design space.
- **Inverse** $\Rightarrow$ three strategies (A/B/C); pick by budget and data.
- **End-to-end** is the point – no single block solves the problem alone.
- **Validate** with FE then experiment. Always.

One-line takeaway

DL doesn't replace physics – it *amortizes* the slow parts into one-time training.

L4 – Handoff

- **Day 3 – Workshops.**
  - W1: FSI surrogate – Lecture 1 pipeline applied.
  - W2: Generative meta-design – Lectures 2 + 4 pipeline on unit cells.
- **Day 4 – Challenge.** Design lattice metamaterial according to required performance.
- **Day 5 – Experiments.** Drop-tower validation.

Message

You have the blocks (L1–L4). The rest of the week is where you assemble them.

Questions?

Tomorrow: Workshops – FSI surrogate + generative meta-design